

MiniSpeedCam - Uploading a Firmware Binary to Bubble

Publishing an over-the-air (OTA) update for the ESP32 or STM32. Assumes the 'Firmware' data type and the 'firmware_check' backend workflow already exist.

The flow: build -> bump version -> compute MD5 -> create a Firmware record (upload the .bin) -> deploy to Live -> test. A device only offers an update when the version in Bubble differs from what it is currently running.

Step 1 - Bump the version, then build

IMPORTANT: bump the version BEFORE you build. If the new version equals what the device already runs, it reports 'ok' and offers no update.

- ESP32: in platformio.ini change FW_VERSION="1.0.0" to the new value (e.g. "1.0.1").
- STM32: in Core/Src/main.c change #define STM_FW_VERSION "1.0.0".

Then build:

```
ESP32:  pio run -e esp32-s3
        -> .pio\build\esp32-s3\firmware.bin

STM32:  pio run          (in the stm32 project)
        -> .pio\build\genericSTM32F301K8\firmware.bin
```

Use firmware.bin -- NOT firmware.factory.bin. The factory image (bootloader + partitions + app) is only for a full wired flash, not OTA.

Step 2 - Compute the MD5

The device verifies the downloaded image against this hash before installing, so a corrupt or truncated download is rejected. In PowerShell:

```
Get-FileHash -Algorithm MD5 .pio\build\esp32-s3\firmware.bin
```

Copy the 32-character Hash value. (Comparison on the device is case-insensitive.)

Step 3 - Create the Firmware record in Bubble

- Bubble editor -> Data -> App data -> All Firmwares -> 'New entry'.
- target = esp (or stm)
- version = the value you set in Step 1 (e.g. 1.0.1)
- md5 = the hash from Step 2
- binary = click to upload, choose firmware.bin

Save the entry. When the file uploads, Bubble stores it (on its S3-backed storage) and the 'binary' field holds the file's URL.

Step 4 - Make sure the URL is a full https:// link

Bubble file URLs are sometimes 'protocol-relative' -- they start with // instead of https://. The device cannot follow a // URL.

- In the firmware_check workflow's 'Return data from API' action, each *_url value must begin with https://. If the raw file URL starts with //, prepend "https:" to it.
- Quick check: paste the file URL into a browser -- it should download the .bin.

Step 5 - Deploy to Live (required)

The device calls the LIVE endpoint (yourapp/api/1.1/wf/firmware_check), never version-test. Development and Live have SEPARATE databases and deploys.

- Create the Firmware record in the LIVE database (switch App data to 'Live'), or it won't be seen.
- If you changed the workflow, also Deploy: top bar -> Deploy current version to Live.

Step 6 - Test before trusting a device

Send a check with fake current versions so a mismatch is guaranteed (use the same Bearer token from the firmware's config.h):

```
$body = @{ camera="YOUR_CAMERA_ID"; esp_fw="0.0.0"; stm_fw="0.0.0" } |  
    ConvertTo-Json  
Invoke-RestMethod -Uri "https://YOURAPP/api/1.1/wf/firmware_check" `  
    -Method Post -Headers @{ Authorization = "Bearer YOUR_TOKEN" } `  
    -ContentType "application/json" -Body $body
```

Expect a response with esp_version, esp_url (https://...), and esp_md5. Then on the device web UI: Device tab -> Firmware -> 'Check for Updates' (status shows the new version) -> 'Install ESP32 Update'.

Checklist

- [] Version bumped BEFORE building
- [] Built firmware.bin (not the .factory.bin)
- [] MD5 computed
- [] Firmware record created (target, version, md5, binary)
- [] File URL is https:// and downloads in a browser
- [] Record exists in the LIVE database; app deployed to Live
- [] firmware_check returns the new version
- [] Device Check + Install works

Common gotchas

- Dev vs Live: the device only sees Live data + deploys. Most 'it didn't work' issues are this.
- 'No update offered': the Bubble version equals the device's. Bump it.
- MD5 mismatch on device: re-hash the exact firmware.bin you uploaded and re-copy it.
- STM32 updates also need the GPIO14 -> BOOT0 wire and a pull-down on BOOT0.